



THE UNIVERSITY OF
MELBOURNE

SWEN90010 - High Integrity Systems Engineering

Alloy Example and Trace-Based Modelling
(Capability-Based Access Control)

Toby Murray



toby.murray@unimelb.edu.au



DMD 8.17 (Level 8, Doug McDonell Bldg)



<http://people.eng.unimelb.edu.au/tobym>



@tobycmurray



THE UNIVERSITY OF
MELBOURNE

NON-EXAMINABLE MATERIAL BEGINS HERE



ACCESS CONTROL

Access Control

- **who** can access **what** in which ways
 - the “who” are called **subjects**
 - e.g. users, processes etc.
 - the “what” are called **objects**
 - e.g. individual files, sockets, processes etc.
 - includes all subjects
 - the “ways” are called **permissions**
 - e.g. read, write, execute etc.
 - are usually specific to each kind of object
 - include those meta-permissions that allow modification of the protection state
 - e.g. own

AC Mechanisms and Policies



THE UNIVERSITY OF
MELBOURNE

- AC Policy
 - Specifies allowed accesses
 - And how these can change over time
- AC Mechanism
 - Implements the policy
- Certain mechanisms lend themselves to certain kinds of policies
 - Certain policies cannot be expressed using certain mechanisms

Protection State

- **Access control matrix** defines the **protection state** at any instant in time

	Obj1	Obj2	Obj3	Subj2
Subj1	R	RW		send
Subj2		RX		control
Subj3	RW		RWX own	recv

Storing Protection State

- Not usually as access control matrix
 - too sparse, inefficient
- Two obvious choices:
 - store individual columns with each object
 - defines the subjects that can access each object
 - each such column is called the object's **access control list**
 - store individual rows with each subject
 - defines the objects each subject can access
 - each such is called the subject's **capability list**

Access Control Lists (ACLs)



THE UNIVERSITY OF
MELBOURNE

Obj1

- Subjects usually aggregated into classes
 - e.g. UNIX: owner, group, everyone
- Meta-permissions (e.g. own)
 - control class membership
 - allow modifying the ACL
- Implemented in almost all commercial OSes

Subj1	R
Subj2	
Subj3	RW

Capabilities

- A **capability** is a capability list element

<i>Subj1</i>	Obj1	Obj2	Obj3	Subj2
	R	RW		send

- **Names** an object to which the capability refers
- **Confers** permissions over that object
- Less common in commercial systems
 - More common in research though

Capabilities: Implementations



THE UNIVERSITY OF
MELBOURNE

- Capabilities must be unforgeable
- On conventional hardware, either:
 - Stored as ordinary user-level data, but unguessable due to sparseness
 - like a password or an encryption key
 - Stored separately (in-kernel), referred to by user programs by index/address
 - like UNIX file descriptors
- Sparse capabilities can be leaked more easily, but are easier to revoke
 - The only solution for most distributed systems

Mandatory vs. Discretionary AC



THE UNIVERSITY OF
MELBOURNE

- Discretionary Access Control:
 - Users can make access control decisions
 - delegate their access to other users etc.
- Mandatory Access Control (MAC):
 - enforcement of administrator-defined policy
 - users cannot make access control decisions (except those allowed by mandatory policy)
 - can prevent untrusted applications running with user's privileges from causing damage

MAC

- Common in areas with global security requirements
 - e.g. national security classifications
- Less useful for general-purpose settings:
 - hard to support different kinds of policies
 - all policy changes must go through sysadmin
 - hard to dynamically delegate only specific rights required at runtime



THE UNIVERSITY OF
MELBOURNE

EXAMINABLE MATERIAL STARTS HERE

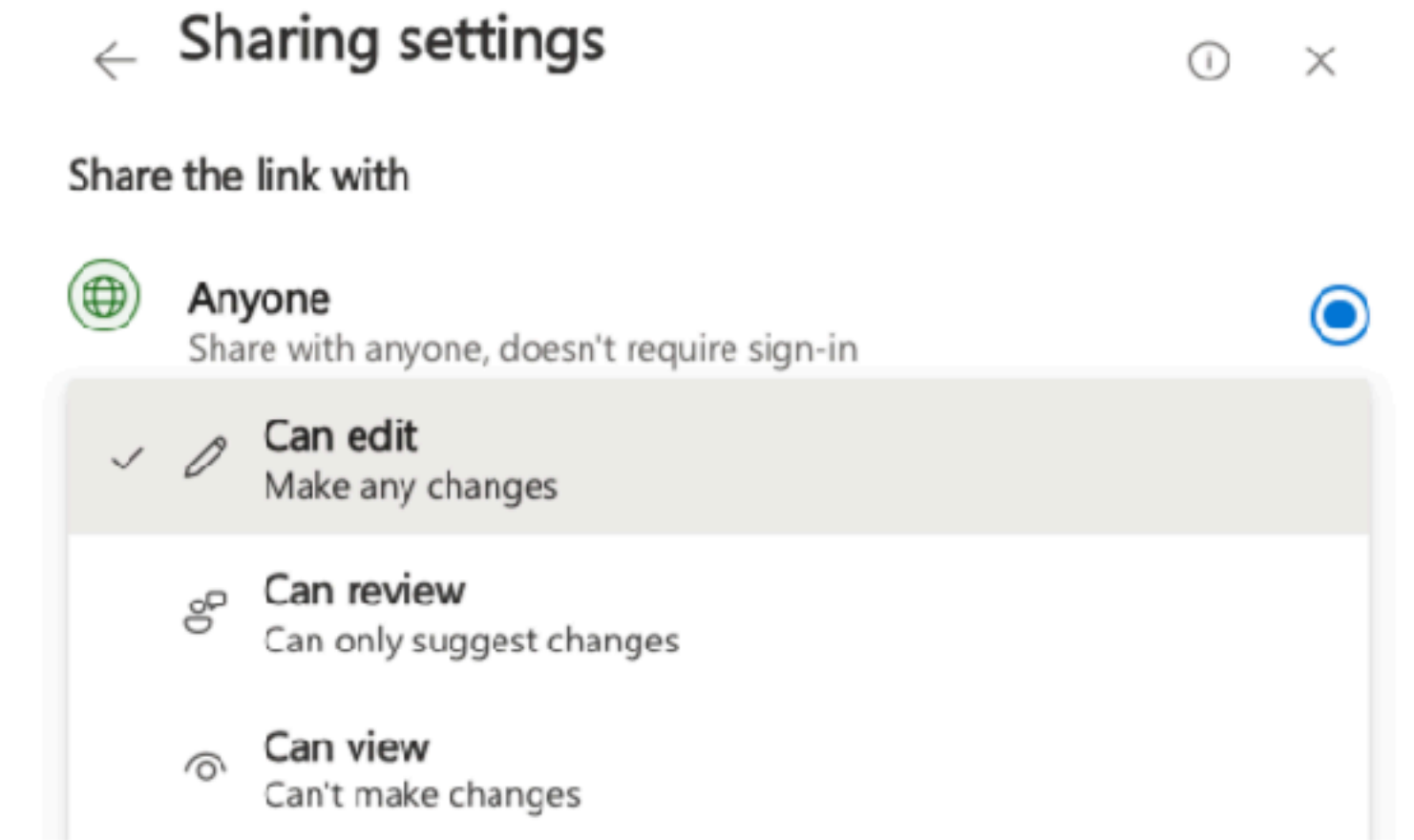
Boebert's Attack



- Boebert 1984: “On the Inability of an Unmodified Capability Machine to Enforce the *-Property”
- Shows an attack on **sparse** capability systems that violates the *-property
 - Where caps and data are indistinguishable
 - Does not work against **partitioned** capability systems
 - Practically all capability-based kernels

Sparse Capabilities

Capabilities as data
e.g. magic links to Google / OneDrive docs etc.



Bell-LaPadula (BLP) Model

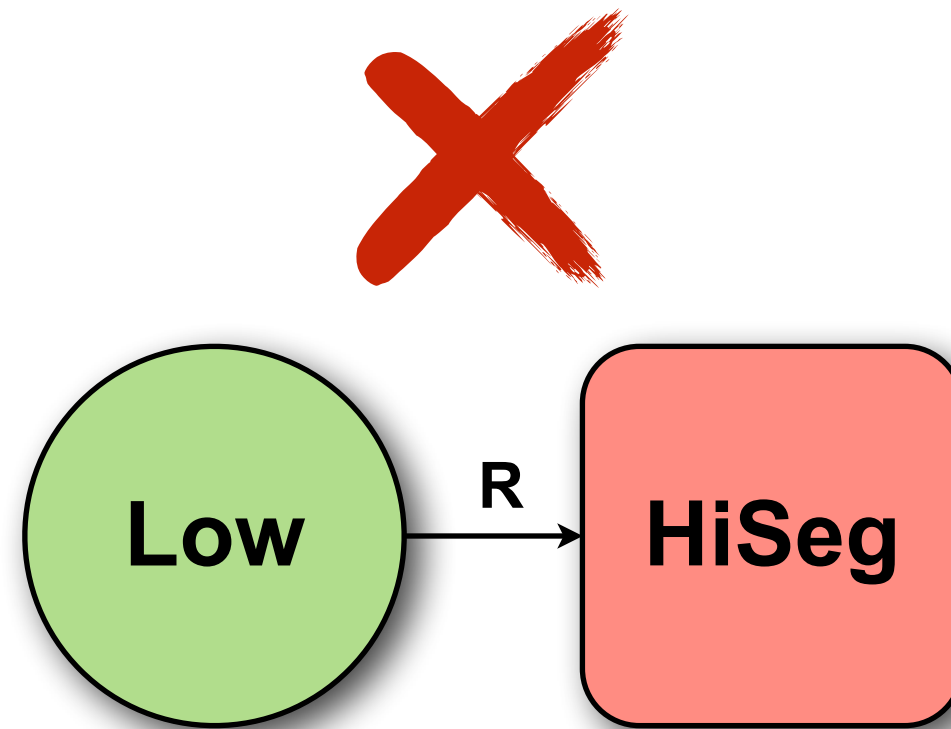


- MAC Policy/Mechanism
 - Formalises National Security Classifications
- Every object assigned a **classification**
 - e.g. TS, S, C, U
- Classifications ordered in a **lattice**
 - e.g. $TS > S > C > U$
- Every subject assigned a **clearance**
 - Highest classification they're allowed to learn

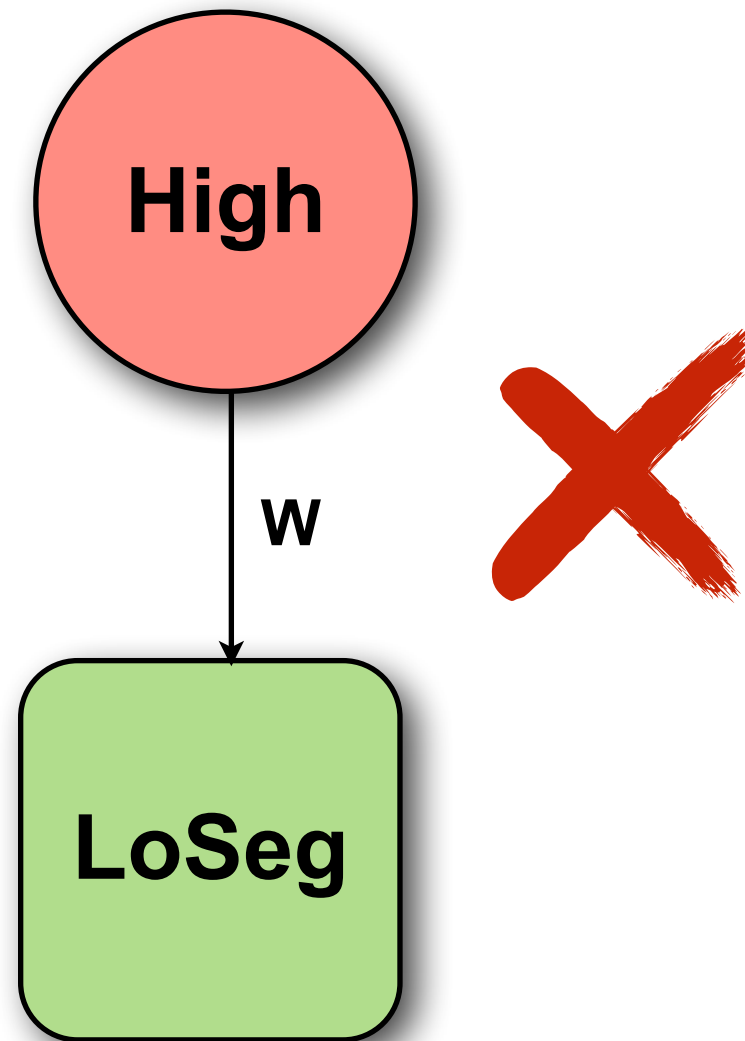
BLP: Rules

- Simple Security Property (“no read up”):
 - s can read o iff $\text{clearance}(s) \geq \text{class}(o)$
 - S-cleared subject can read U,C,S but not TS
 - standard confidentiality
- *-Property (“no write down”):
 - s can write o iff $\text{clearance}(s) \leq \text{class}(o)$
 - S-cleared subject can write TS,S, but not C,U
 - to prevent accidental or malicious leakage of data to lower levels
-

No Read Up



No Write Down



Boebert's Attack



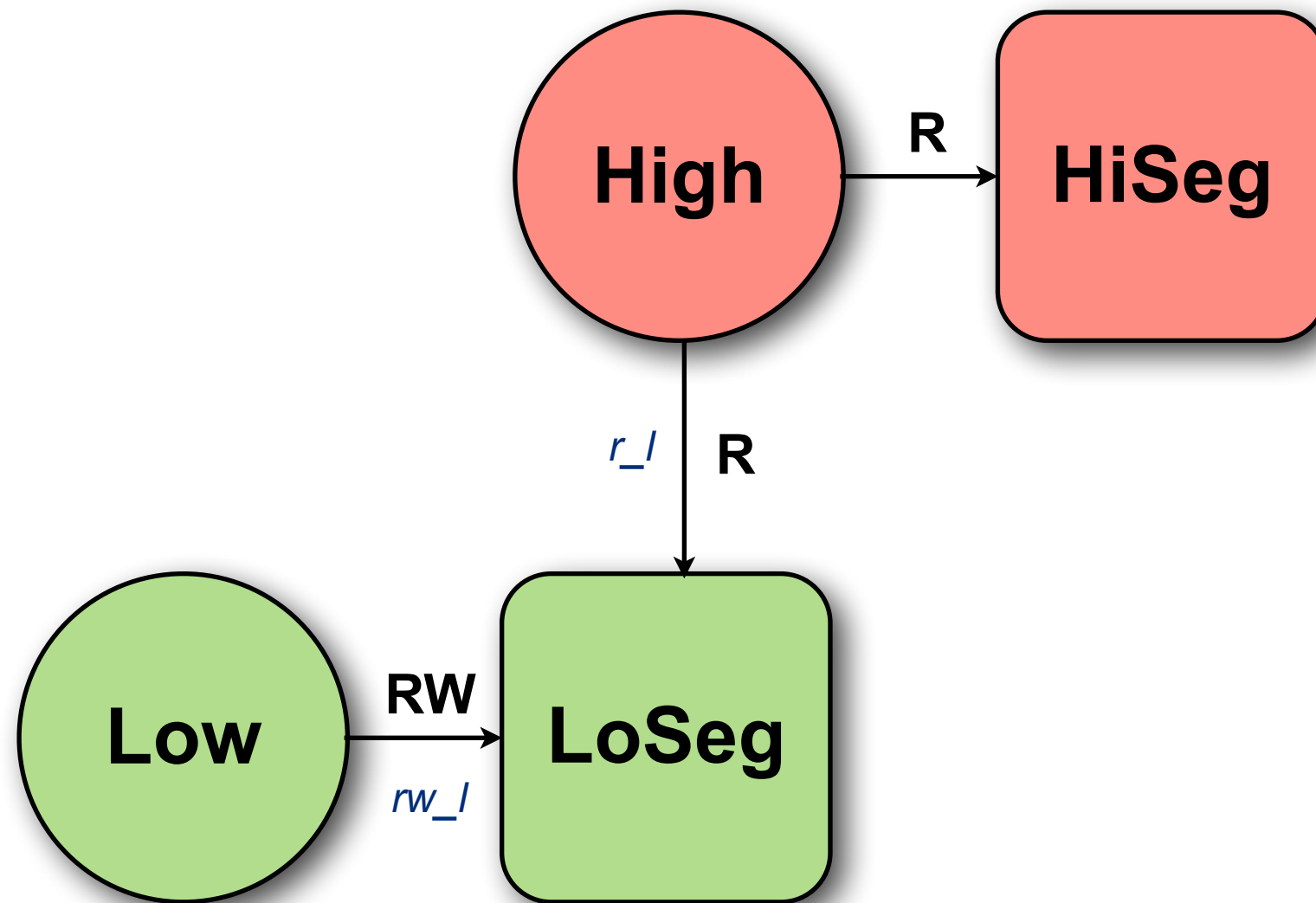
- Boebert 1984: “On the Inability of an Unmodified Capability Machine to Enforce the *-Property”
- Shows an attack on **sparse** capability systems that violates the *-property
 - Where caps and data are indistinguishable
 - Does not work against **partitioned** capability systems
 - Practically all capability-based kernels



THE UNIVERSITY OF
MELBOURNE

LET'S MODEL THIS IN ALLOY

Initial Conditions for Attack



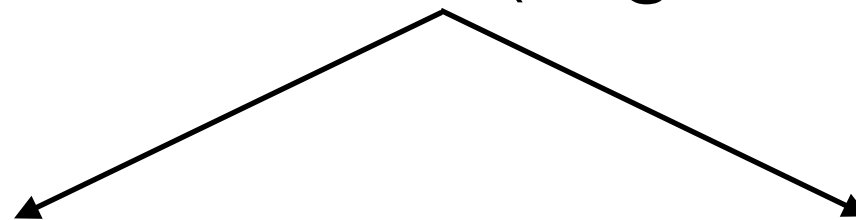
What we need to model

Data



Capabilities: each points to an **object** and carries certain **access rights** (permissions)

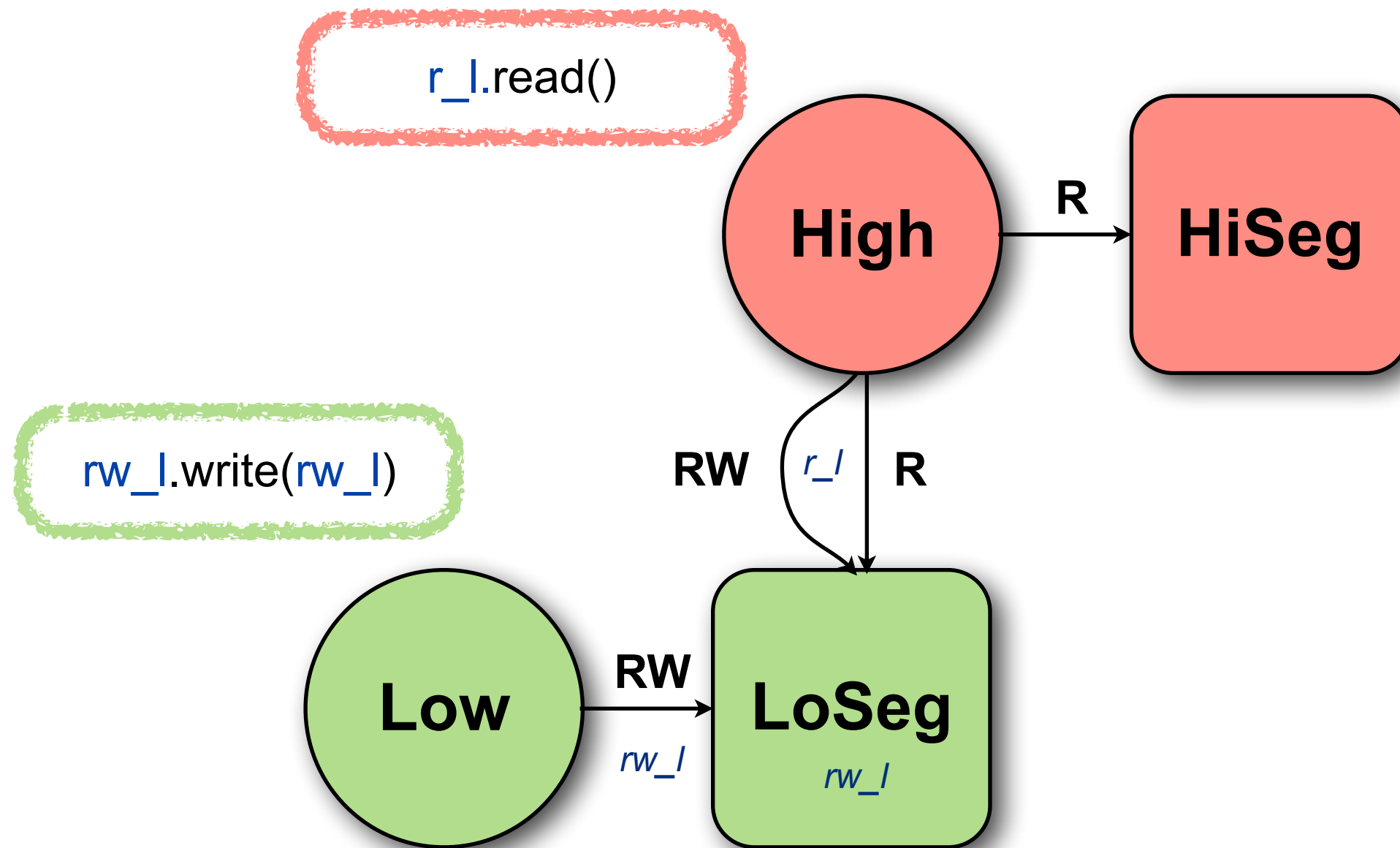
Objects: things capabilities point to; each has a **classification** (High or Low)



Actors (i.e. programs): that can use capabilities that they **possess**

Memory Segments: which **store** Data

Boebert's Attack



- **Low** writes his cap into the low segment
 - from which **High** reads it out

Boebert's Attack: Lessons



- Not all mechanisms suited to all policies
- Many policies treat data- and access-propagation differently
 - BLP is one example
 - Cannot be expressed using sparse capability systems
- This does **not** mean that capability systems and MAC are incompatible in general



TRACE-BASED MODELLING IN DETAIL

Trace-Based Modelling

Can't just consider **individual** state transitions:

```
pred read [s : State]
```

```
pred write [s : State]
```

```
pred state_transition [s : State] {  
  (read [s] or write [s] or state_unchanged[s])  
}
```

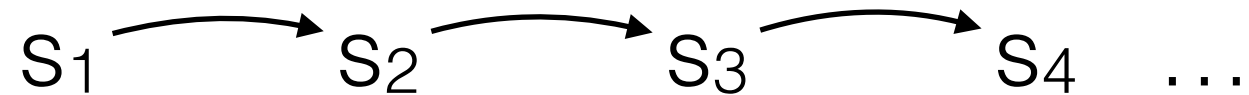
$s \rightarrow s'$

We need to talk about a **sequence**
(aka a **trace**)
of such transitions.

Trace (Sequence of States)



THE UNIVERSITY OF
MELBOURNE

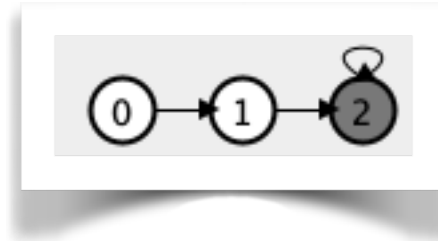


There is a state transition between each pair of states in the sequence

How do we define this in Alloy?

Traces in Alloy

Alloy 6 **natively** reasons about traces



The models (behaviours) that Alloy considers
(e.g. when doing **check** and **run**)
are **lasso traces**

```
sig Something {}
```

```
one sig State {
```

```
  var field: Something
```

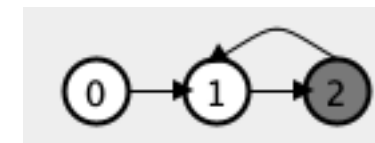
```
}
```

```
pred next_next_is_same_as_next {
```

```
  State.field" != State.field and State.field' = State.field
```

```
}
```

```
run next_next_is_same_as_next
```



Defining Traces

We need to give these traces **meaning**

Constrain the **state transitions**:

fact trans { **always all** s: State |
op1[s] **or** op2[s] **or** ... **or** unchanged[s] }

(Often, also) constrain the **first state**:

fact init { **all** s: State | init[s] }